

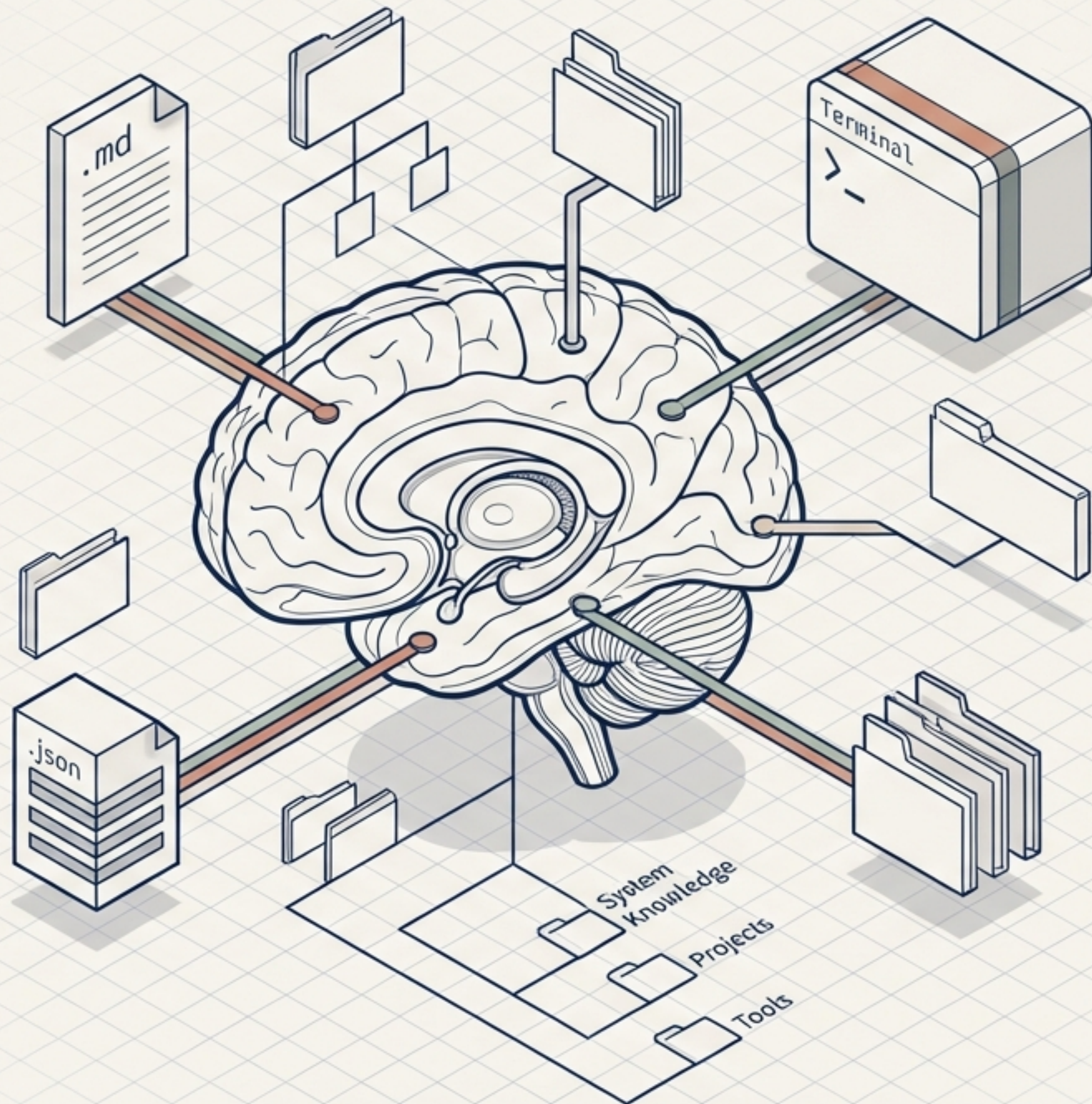
# 掌握 Claude Code： 打造你的 AI 第二大腦

從本地終端機到全自動化商業引擎——  
4 小時精華系統拆解與架構藍圖

系統架構

自動化

生產力倍增





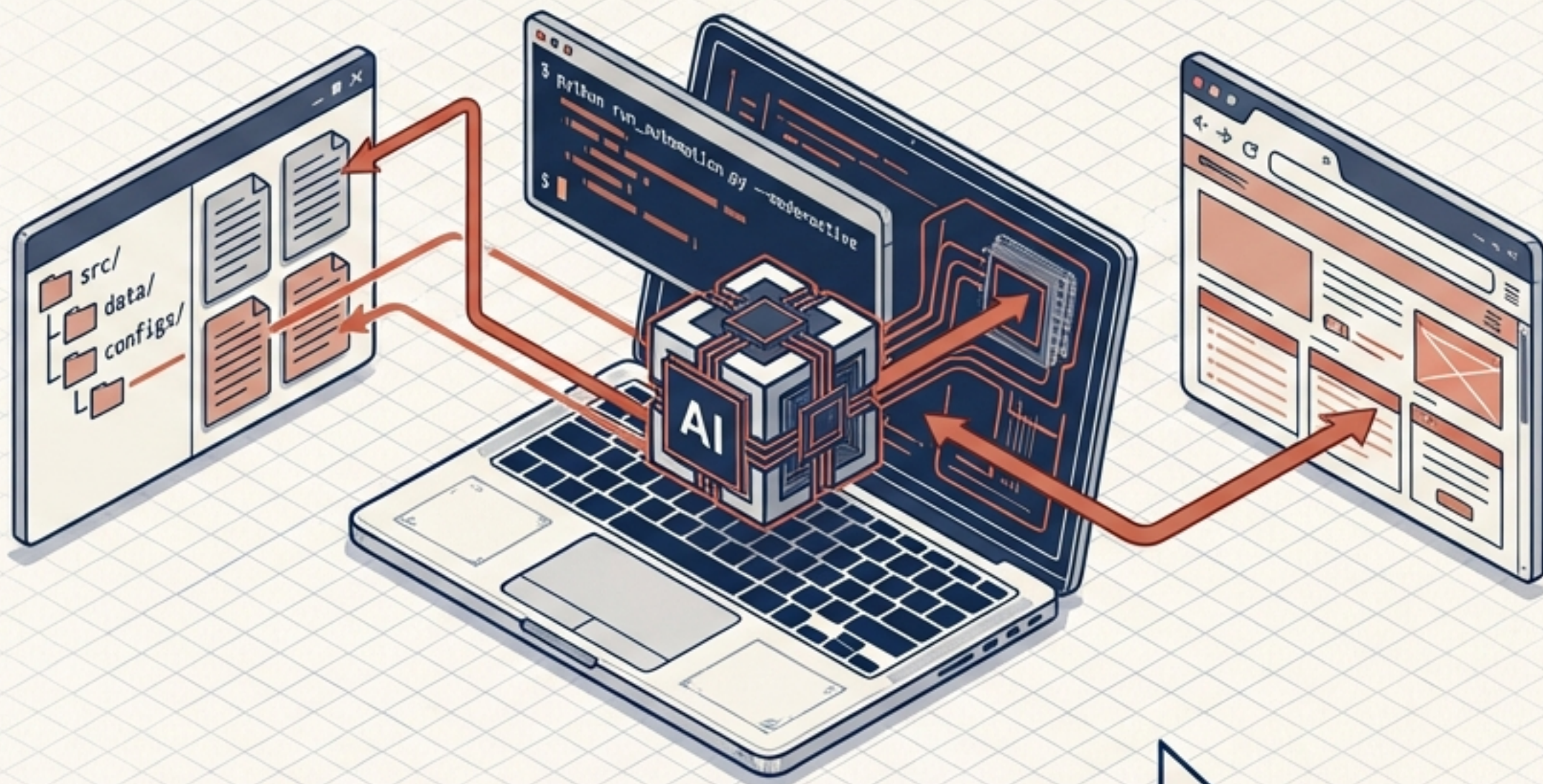
# 核心典範轉移：它不是對話機器人，而是「執行引擎」

雲端對話框：複製貼上的  
勞力密集模式



本地執行引擎：直接讀寫、除錯與操控

Directly read/write, debug, and control



核心洞察：將 AI 視為一位長駐在你電腦底層底層的資深工程師與專案經理。它能自主讀寫檔案、執行終端機指令、甚至開啟瀏覽器爬取資料。這是實現年獲利 400 萬美元業務自動化的技術基石。



# 開發環境解剖 (Anatomy of the Workspace)

整合開發環境 (IDE) 本質上只是「檔案總管 + 記事本 + ChatGPT」的結合體。拋棄對終端機的恐懼，這是你的新工作檯。

**Terminal 模式解鎖極限**：直接在終端機執行 Claude 能解鎖「Fast Mode (2.5倍速執行)」與「平行處理」，讓你同時啟動多個 AI 代理。

Terminal

```
~/project > claude run optimize.py --fast  
~/project >
```

檔案與資料夾結構  
(File Explorer)

檔案與資料夾結構  
(File Explorer)

- src/
  - components/
    - App.jsx
  - styles.css
  - README.md

```
App.jsx  
1 import React from 'react';  
2  
3 const App: React.FC = () => {  
4   const [count, setCount] = React.useState(0);  
5  
6   return (  
7     <div className="container">  
8       <h1>AI-Driven Development</h1>  
9       <p>Count: {count}</p>  
10    </div>  
11  );  
12  };  
13  
14  export default App;  
15
```

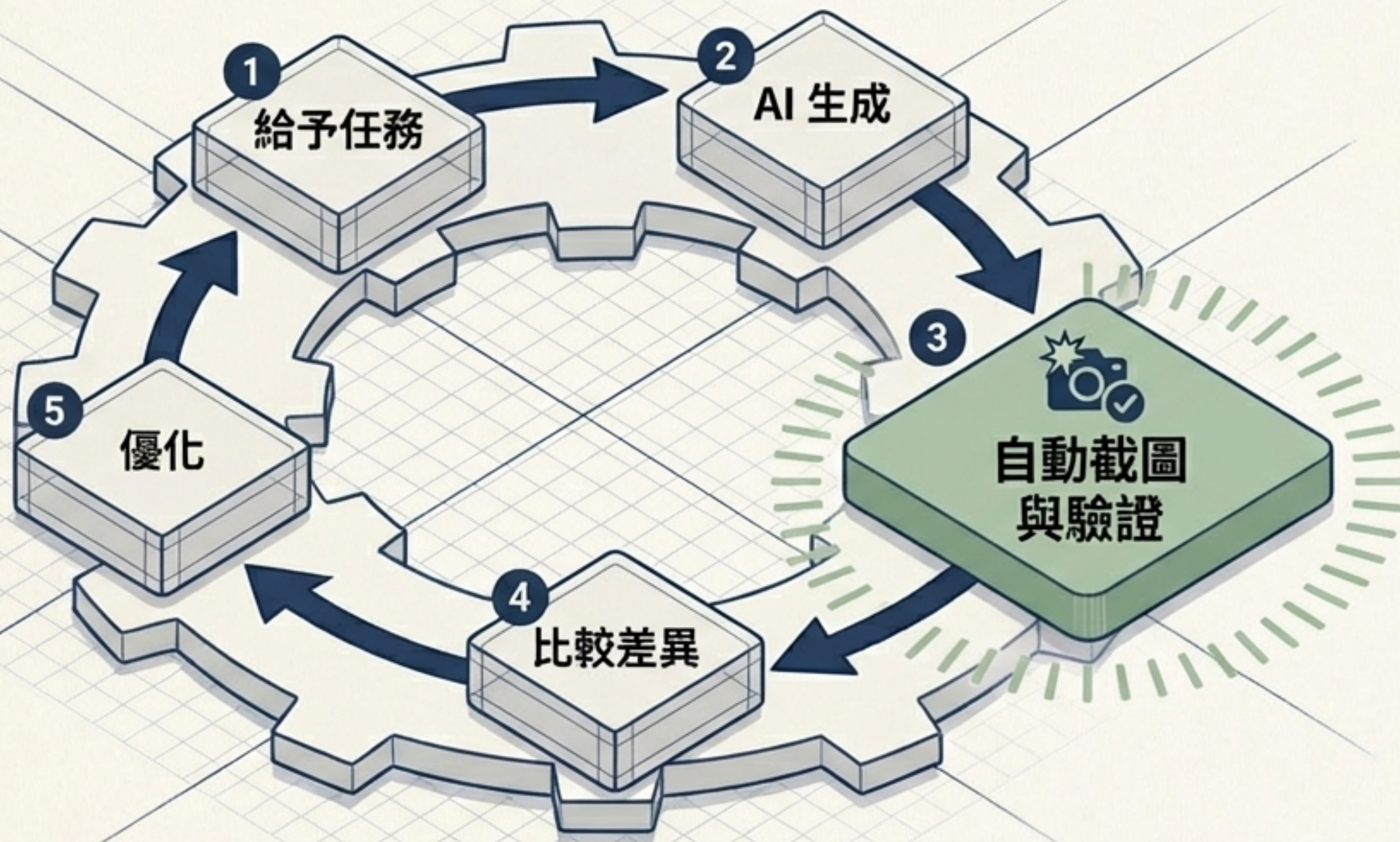
程式碼與文本編輯區  
(Text Editor)

AI 代理指令區  
(Agent Chat Widget)

請優化 App.jsx 的效能。  
已分析，建議使用 React.memo  
包裹組件，並檢查 useEffect  
的依賴項。



# AI 建造法則：Test-Do-Verify 迴圈



**人類開發：**

追求 1 次到位  
= 耗時 5 小時

**Claude Code：**

迭代 5 次到達完美  
= 耗時 5 分鐘

核心洞察：缺乏「驗證 (Verify)」環節，是多數人認為 AI 產出不佳的根本原因。給予 AI 截圖比對或自動化測試的能力，讓它極速自我修正。



# 複雜專案的秘密武器：Plan Mode

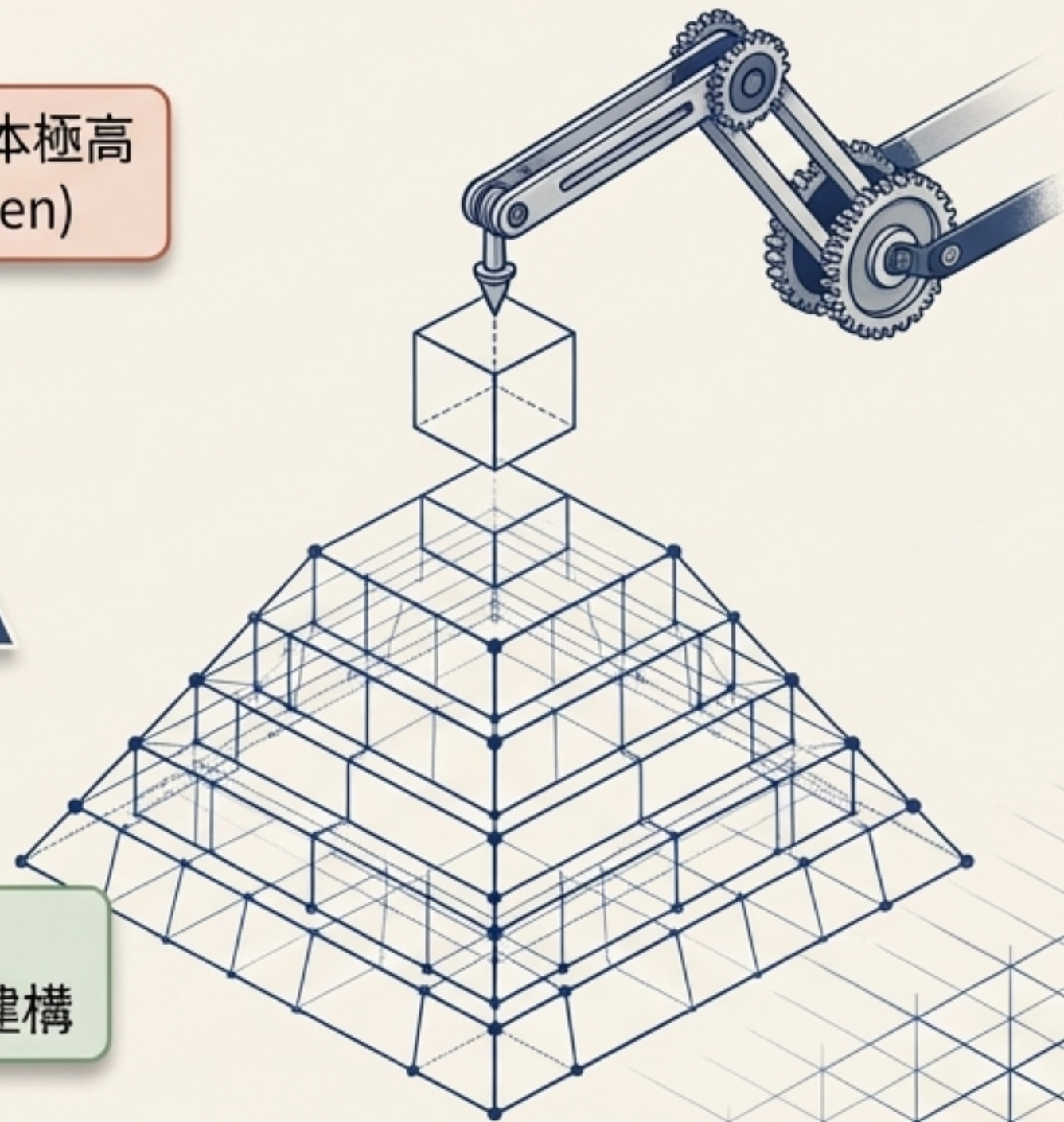
在執行複雜架構前，強迫進入 Plan Mode。Claude 會進行唯讀探索、產出架構藍圖。  
在設計圖階段修改，遠比在實體程式碼中除錯便宜且快速。



**無計畫建構：**重工成本極高  
(浪費時間與高額 Token)



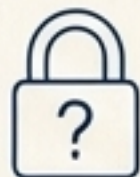
**Plan Mode (計畫模式)：**  
1 分鐘的規劃 = 節省 10 分鐘的建構





# 權限模式診斷矩陣 (Permission Modes)

掌控 AI 的自由度。根據任務的風險層級與對專案的熟悉度，動態切換權限模式。



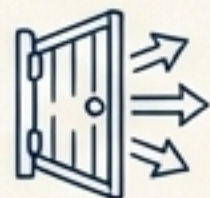
## Ask Before Edits (安全模式)

預設值。每次修改檔案皆需授權。適合極高風險環境。



## Edit Automatically (自動編輯)

自動修改現有檔案，但新建檔案需詢問。適合日常專案維護。



## Bypass Permissions (無限制模式)

賦予 AI 完全的讀寫與創建控制權。最高效率，適合知識工作者與進階開發。（警告：需承擔誤刪檔案風險）

10x 效率首選

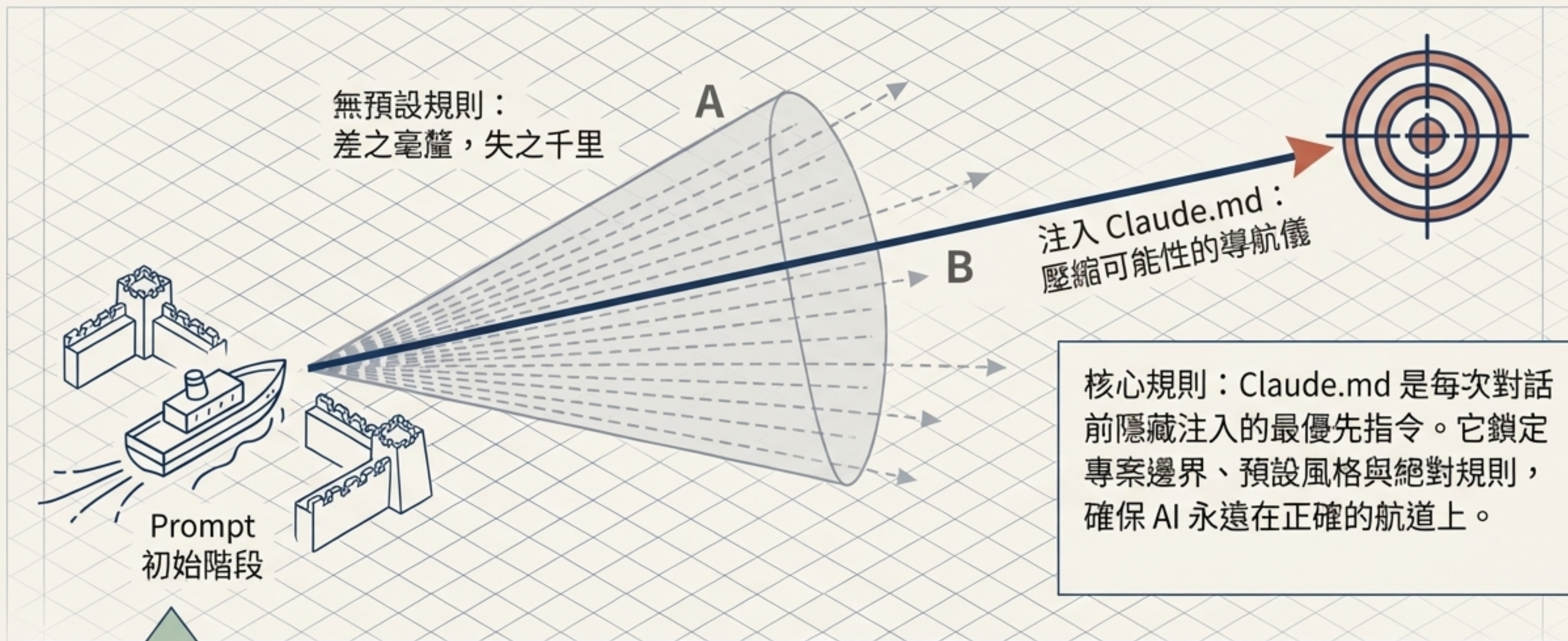


## Plan Mode (唯讀探索)

不修改任何檔案，專注於規劃與推理。



# 系統大腦：Claude.md 的航向控制

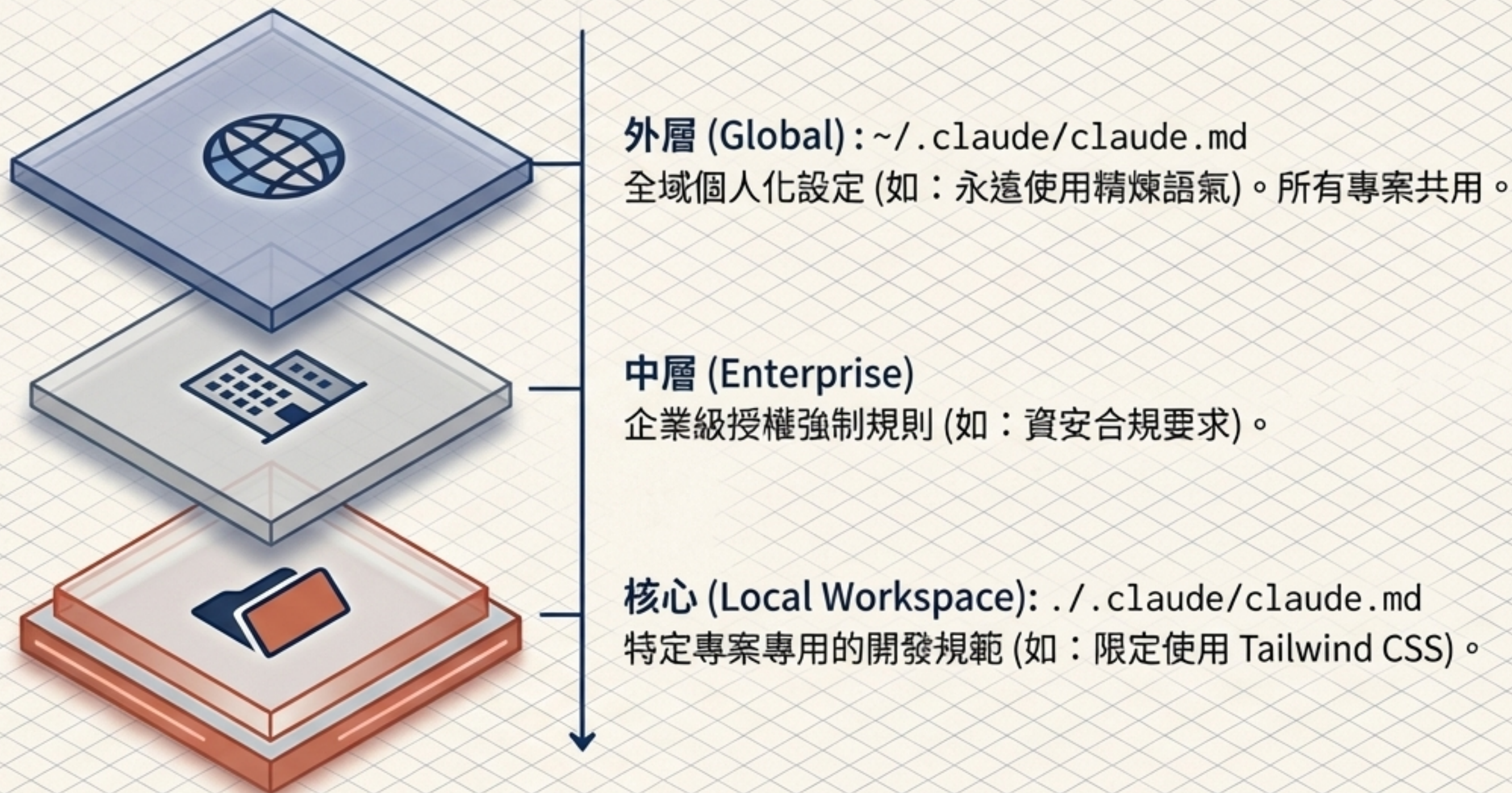


**Actionable Tip:** 在新資料夾輸入 `\init`，讓 Claude 自動為你生成專屬的 Claude.md。



# 大腦的層次：三層作用域 (The Three-Tier Brain)

不要將所有規則塞在同一個檔案裡，那會消耗過多 Token。高層級規則會覆蓋低層級規則，實現「個人化大腦」與「專案執行腦」的完美分離。





# 架構深度解剖：.claude 隱藏目錄

為了避免單一 Claude.md 體積失控，進階玩家會利用隱藏的 .claude 目錄進行模組化拆分。當你解鎖了這個目錄，你就是「AI 系統架構師」。

## /.claude (隱藏目錄)

### /rules

將規範細分（如 design.md、security.md），避免單一檔案過於龐大。

### /skills

自定義的自動化腳本與 SOP 檢查清單。

### /agents

具備專屬記憶與特定工具權限的子代理人。

### memory.md

跨對話的全局記憶檔（自動儲存上下文）。



# 擴展大腦的能力矩陣 (The Capability Matrix)

當內建功能不足時，如何精準選擇擴充方式？這是一張價值連城的決策表。

| 擴充工具                                                                                                 | 功能 (Function)                  | Token 消耗                 | 設定難度     |
|------------------------------------------------------------------------------------------------------|--------------------------------|--------------------------|----------|
| System Tools (內建工具)                                                                                  | Bash, Web Fetch, 讀寫檔案          | 中 (~17k tokens)          | 零 (開箱即用) |
| Subagents (子代理)                                                                                      | 隔離上下文的獨立 AI<br>(如研究員、審查員)      | 高 (但保護主代理)               | 中        |
| Skills (技能)                                                                                          | 自定義 SOP 與 Python 工作流腳本         | 極低 (<100 tokens)         | 高        |
|  MCP<br>(模型上下文協議) | 串接第三方軟體<br>(如 Chrome, ClickUp) | 極高<br>(>20k tokens, 易臃腫) | 低        |

**Insight:** 濫用 MCP 會讓 AI 變笨且變貴；高頻任務應盡量轉化為 Skills。



# 子代理的隔離藝術 (The Subagent Strategy)

為什麼需要 Subagents？為了對抗「上下文污染」。呼叫零上下文的 QA 子代理進行交叉驗證，能以極低成本換取最高品質除錯。

## 糟糕的做法 (Context Pollution)



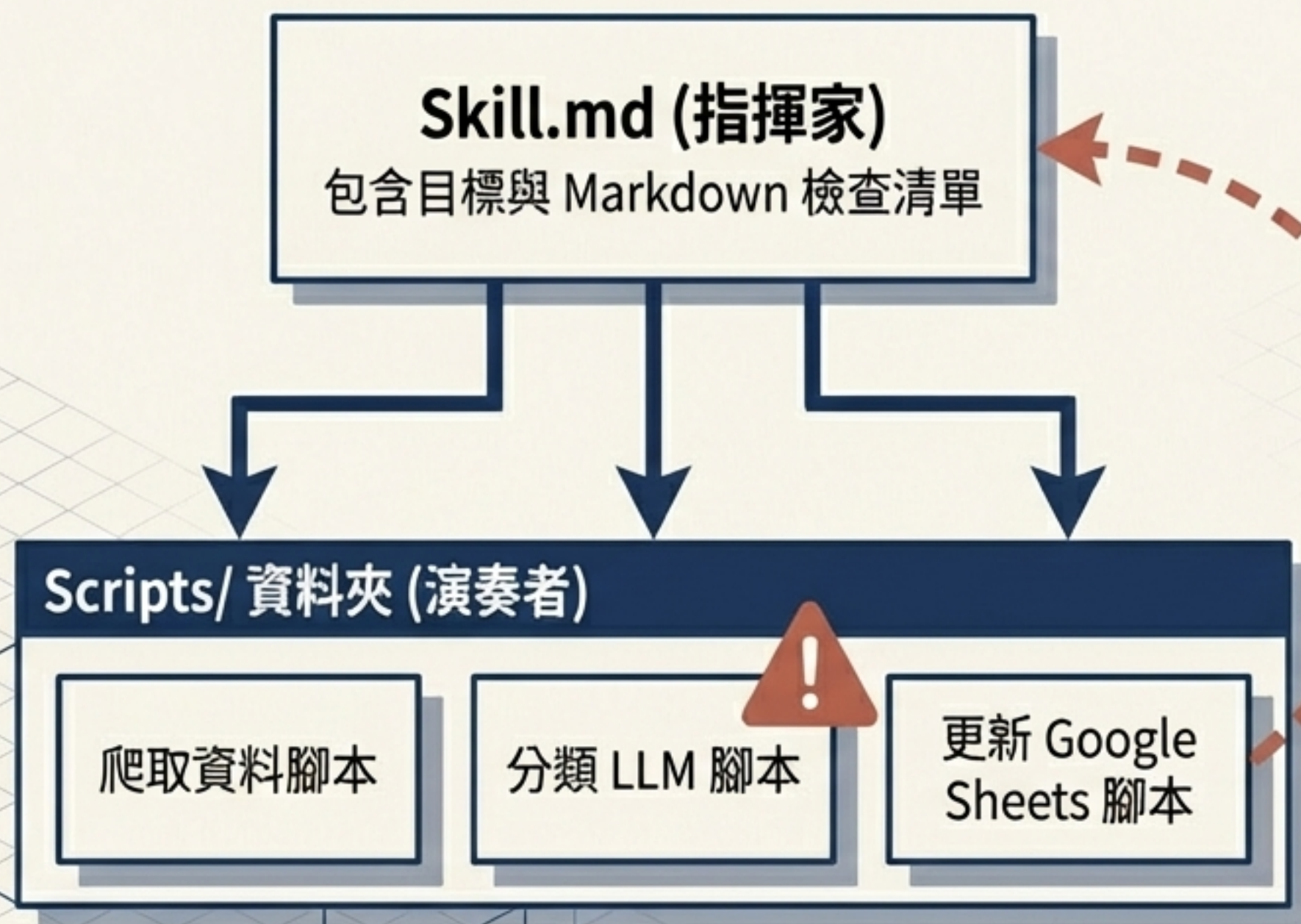
## 卓越的做法 (Subagent Strategy)





# 最高經濟價值：技能腳本化 (Economic Leverage)

將日常知識工作轉化為自動化技能。Claude 會啟動「編排修正機制」自動除錯，徹底取代代脆弱的傳統腳本。



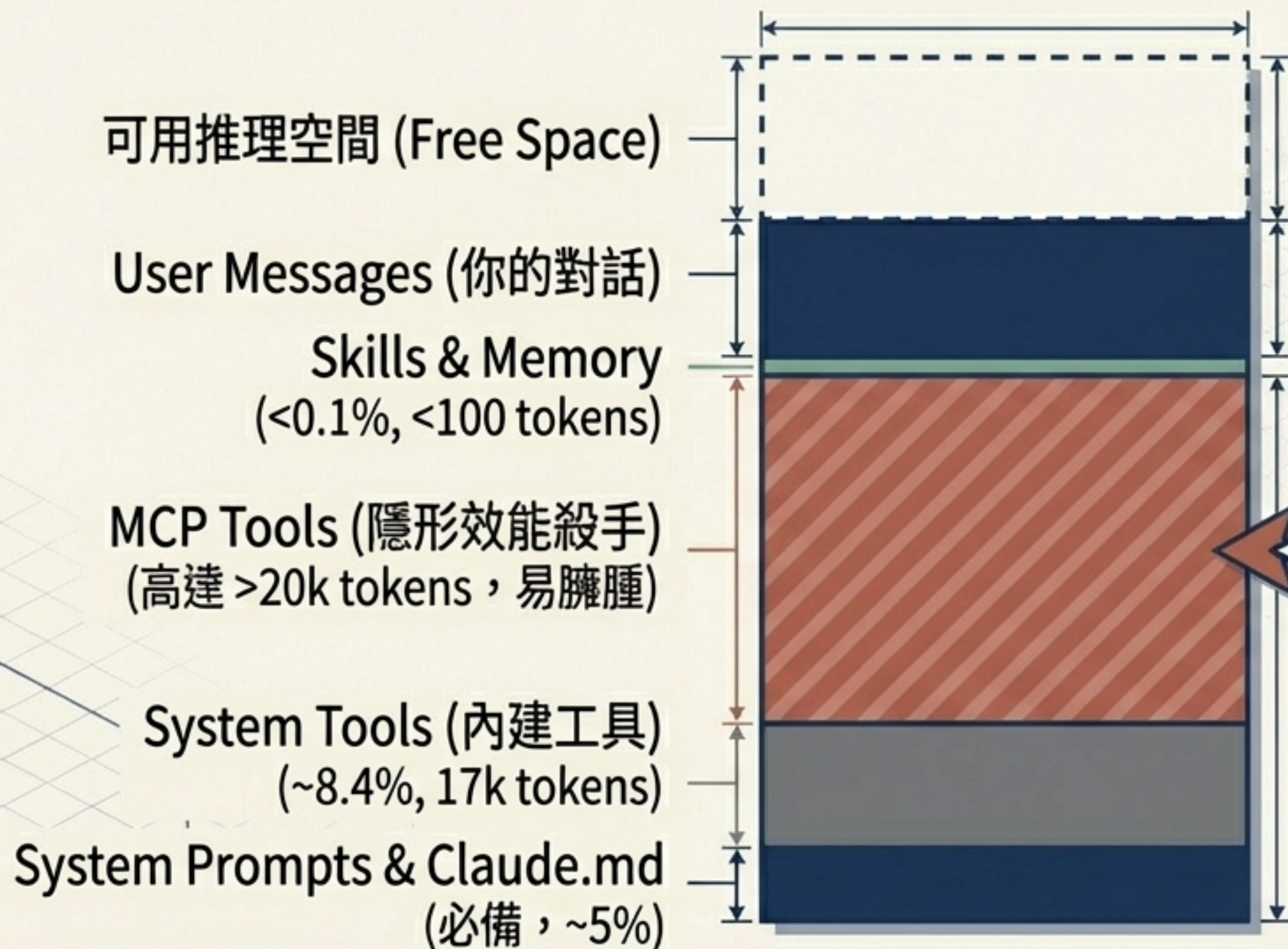
**反饋迴圈 (Orchestration)：**  
自動閱讀 API 錯誤並重寫腳本，無需人類介入

**1,000 筆潛在客戶名單**  
傳統手動需 2 小時 -> 技能自動化僅需 **87 秒**



# 破解 Token 經濟學 (Context Management Economics)

你的 200,000 Tokens 都花在哪裡了？看懂這個，才能真正掌控 AI 的運作成本與智商。

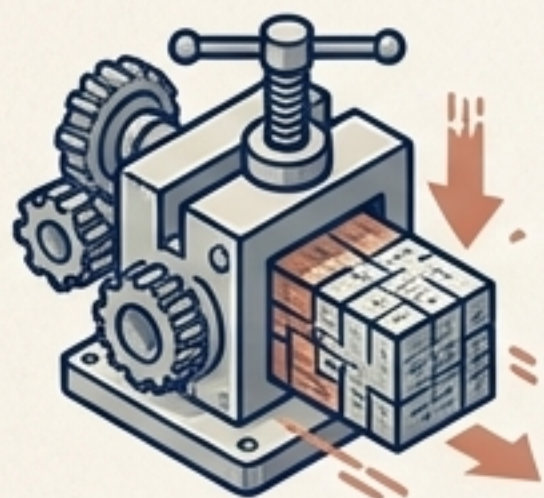


**Insight:** 亂裝臃腫的 MCP 會吃掉最高品質的上下文開端空間，讓 AI 變笨且 API 費用暴增。



# 對抗上下文腐敗 (Beating Context Rot)

隨著對話拉長，AI 就像人類一樣會遺忘細節並產生幻覺。熟練運用這套工具箱，確保模型始終保持最高資訊密度的專注狀態。



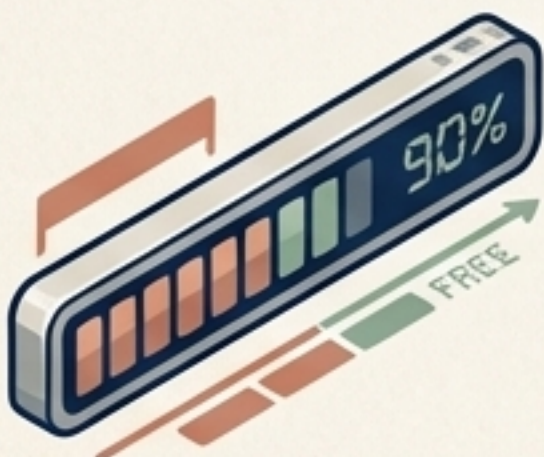
## Auto-Compaction (自動壓縮)

Claude 隱藏的機制，當快觸及上限時，會將過往對話自動壓縮為高資訊密度的摘要。



## 手動壓縮 (/compact)

主動清理雜訊的終端機指令。提煉當前對話上下文，釋放寶貴的 Token 空間。



## 監控儀表板 (/status line)

在終端機加入動態 Token 使用量進度條，避免消耗盲點。



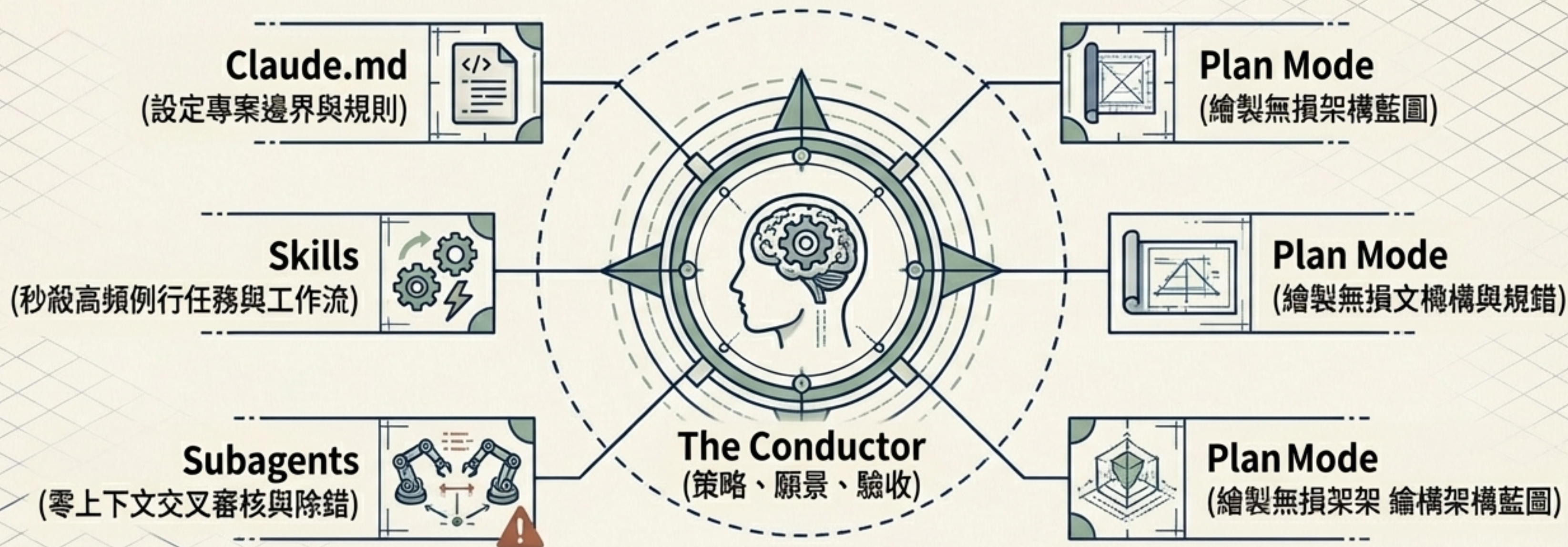
## 延伸思考 (Extended Thinking)

將耗時的推理轉移至隱藏的草稿區，不會永久污染上下文歷史紀錄。



# 終極視角：你不再是打字員，你是指揮家

Claude Code 不是用來取代你的打字速度，而是將你升級為擁有無限助理的專案經理與系統架構師。



「將雙手交給 AI，將大腦留給策略。」

(Give your hands to AI, keep your brain for strategy.)